

Task — MCP Client-Server Package

1.1 Background

The Model Context Protocol (MCP) standardizes how AI agents discover and call external tools through a JSON-RPC 2.0 message layer. Rather than hardcoding tool integrations into a single agent script, MCP separates concerns: the server owns the tools and their schemas, the client (agent) discovers and invokes them at runtime.

Your MCP server will expose 3 tools related to maritime regulation queries as described below

1. `sire_lookup` - Look up a SIRE 2.0 question by reference ID or keyword search across chapter titles and question text.
2. `marpol_check` - Given a vessel activity description (e.g. 'discharging bilge water 15nm from coast'), look up applicable MARPOL Annex rules and return compliance requirements.
3. `ism_requirement` - Retrieve ISM Code (International Safety Management) requirements for a given SMS element or operation type.

The data.zip includes the synthetic data to use for this task.

1.2 Implementation Requirements

The server:

1. Tools must be registered via a `@tool` decorator or explicit registry.
2. Transport: implement stdio transport (required) and SSE transport (optional)
3. Tool discovery: the server must respond to `tools/list` requests with JSON-RPC 2.0 format
4. Error handling: return structured JSON-RPC error objects for invalid tool names, validation failures, and internal errors.

The agent:

1. Use LangGraph to implement a ReAct loop. Each tool call goes through the MCP client, not a direct function call.
2. Use any LLM you have API access to (OpenAI, Anthropic, etc.) as the reasoning backbone, or locally hosted open source LLM.

The client:

3. On startup, connect to the MCP server, call `tools/list` and build a local tool registry mapping tool names to their input schemas.
4. Expose a `call_tool()` async method that sends a `tools/call` request and returns the parsed result and a `list_tools() -> list[ToolDefinition]` method returning the discovered tool schemas.
5. Handle MCP server errors gracefully

1.3 Deliverables

1. Use the following queries to run end-to-end test of your implementation and copy answers in one file `query_output.json`.
 - A vessel is 8nm offshore and needs to discharge bilge water.
 - The SIRE inspector also asked about the passage plan during the last inspection.
 - What MARPOL rules apply, and which SIRE question should the Chief Officer review?
 - We are in the Baltic Sea and need to discharge treated sewage. What MARPOL restrictions apply, and which SIRE chapter would an inspector use to assess our compliance records?
 - We are transiting the North Sea ECA and our bunker delivery note shows 0.08% sulphur fuel. The SIRE inspector also wants to see our VDR annual performance test certificate and our last enclosed space entry permit. Are we MARPOL compliant, and which SIRE questions cover the other two findings?
2. Package your implementation and `query_output.json` from 1 into one zip file which is installable with `pip install -e .`